# **GPRA**: Enhancing Small Model **Reasoning** Under Resource Constraints via LoRA Policy Optimization with Implicit Process Rewards

Yifan Li, Lee Nara, Chengzhi Zhang, Fandi Meng, Ruiqi Zhu
Nanyang Technological University
LIYI0115@e.ntu.edu.sg

**Abstract**

Reinforcement learning with verifiable rewards (RLVR) has emerged as a powerful paradigm for enhancing the reasoning capabilities of large language models (LLMs). However, most existing approaches demand substantial computational resources, limiting accessibility for researchers with constrained GPU budgets. In this work, we investigate a resource-efficient approach to improving mathematical reasoning in small language models by combining two recent innovations: (1) LoRA-based parameter-efficient RL training, as demonstrated by the Tina framework, and (2) online implicit process reward models (PRMs) from the PRIME framework. We term our combined method **GPRA** (**G**RPO + **P**RIME + Lo**RA**), which applies LoRA for the policy model update while maintaining a full-parameter implicit PRM that is updated online. Starting from `DeepSeek-R1-Distill-Qwen-1.5B`, we first reproduce a LoRA+GRPO baseline on 2×A100-80GB GPUs, then augment it with GPRA. Our method achieves a best average accuracy of **52.19%** across four mathematical reasoning benchmarks (AIME 2024, AIME 2025, AMC 2023, MATH-500), representing a **4.73 percentage point improvement** over the LoRA+GRPO baseline (47.46%). Notably, GPRA reaches higher peak performance in fewer training steps, demonstrating improved sample efficiency consistent with the findings of PRIME. Our results validate that dense process rewards can meaningfully benefit parameter-efficient RL training on small models under tight resource constraints, offering a practical recipe for researchers with limited compute. All code, training logs, and model weights are publicly available.

- Code Repository: https://github.com/LYF22034/open-rs2-GPRA
- Model Weights and Checkpoints: https://huggingface.co/whalexdfsa/open-rs2-GPRA

## 1 Introduction

The development of reasoning capabilities in language models has undergone a paradigm shift with the advent of reinforcement learning with verifiable rewards (RLVR). Pioneered by models such as OpenAI o1 [OpenAI, 2024] and DeepSeek-R1 [DeepSeek-AI, 2025], this approach enables models to discover effective reasoning strategies through direct interaction with reward signals derived from ground-truth verification, rather than merely imitating expert demonstrations. The resulting models exhibit substantial improvements in complex multi-step reasoning tasks, particularly in mathematics and coding.

However, the computational cost of RL-based reasoning training remains a significant barrier. State-of-the-art pipelines typically require dozens of high-end GPUs, extensive rollout budgets, and full-parameter fine-tuning of models with tens of billions of parameters. This raises a fundamental question that motivates our work:

***How can we effectively enhance mathematical reasoning in small language models when GPU resources are severely limited?***

Recent work has begun to address different aspects of this challenge. The Tina framework [Wang et al., 2025] demonstrated that applying LoRA [Hu et al., 2021] during GRPO-based RL training on a 1.5B parameter model can achieve reasoning performance competitive with full-parameter training at a fraction of

the cost ($9 USD for the best checkpoint). Meanwhile, the PRIME framework [Cui et al., 2025] showed that incorporating dense token-level rewards through implicit process reward models (PRMs) can yield 2.5× sample efficiency gains and substantial performance improvements over sparse outcome-only rewards, while requiring only outcome-level labels for PRM training.

In this work, we combine these two innovations into a unified, resource-efficient pipeline that we call **GPRA** (**G**RPO + **P**RIME + Lo**RA**). GPRA applies LoRA for the policy model update during GRPO training, while maintaining a full-parameter implicit PRM that provides dense token-level rewards and is updated online alongside the policy. Specifically, we:

1. **Reproduce the LoRA+GRPO baseline** from the Tina framework on 2×A100-80GB GPUs, establishing a baseline for parameter-efficient RL reasoning.
2. **Integrate an online implicit PRM** with full-parameter updates that provides dense token-level process rewards, following the core mechanism of PRIME.
3. **Demonstrate consistent improvement**: GPRA achieves a 4.73% average accuracy improvement over the LoRA+GRPO baseline across four mathematical reasoning benchmarks, with improved sample efficiency and higher peak performance on competition-level problems.

Our approach requires only two A100-80GB GPUs—one dedicated to vLLM inference for rollout generation and the other to RL training (policy LoRA update and full-parameter PRM update). To the best of our knowledge, this is the first work to explore the combination of LoRA-based policy optimization with full-parameter online implicit process rewards for small model reasoning enhancement.

In service of accessibility and open research, we publicly release all code, training logs, and model weights:

# 2 Related Work

## 2.1 RL for LLM Reasoning

Reinforcement learning has been widely adopted for enhancing reasoning capabilities in LLMs. OpenAI o1 [OpenAI, 2024] first demonstrated the potential of large-scale RL for reasoning, and DeepSeek-R1 [DeepSeek-AI, 2025] showed that GRPO with verifiable rewards can train models to exhibit emergent reasoning behaviors. Subsequent open-source efforts—including SimpleRL [Zeng et al., 2025], DeepScaleR [Luo et al., 2025], STILL [Min et al., 2024], and Open-RS [Dang and Ngo, 2025]—have explored various strategies for efficient RL-based reasoning.

## 2.2 Parameter-Efficient RL Reasoning

The Tina framework [Wang et al., 2025] pioneered the use of LoRA during RL training for reasoning models. By applying parameter-efficient updates to a 1.5B base model (`DeepSeek-R1-Distill-Qwen-1.5B`), Tina achieved reasoning performance competitive with full-parameter SOTA models at dramatically lower cost. The authors hypothesized that LoRA's effectiveness stems from rapidly adapting the model to the structural format of reasoning rewarded by RL, while preserving the base model's underlying knowledge. Their best model achieved 43.33% Pass@1 on AIME 2024 and 50.60% average across six benchmarks.

## 2.3 Dense Process Rewards and PRIME

While sparse outcome rewards provide binary correctness signals only at the end of a response, dense process rewards offer feedback at each intermediate reasoning step, improving credit assignment and sample efficiency [Lightman et al., 2023, Wang et al., 2023]. However, training process reward models (PRMs) traditionally requires expensive step-level annotations that are prohibitively costly to obtain at scale.

PRIME (Process Reinforcement through Implicit Rewards) [Cui et al., 2025] addresses this challenge through implicit process reward modeling [Yuan et al., 2024]. The key insight is that a language model trained with only outcome-level labels can be repurposed as a token-level PRM by computing:

$$r_\phi(y_t) := \beta \log \frac{\pi_\phi(y_t|y_{<t})}{\pi_{\text{ref}}(y_t|y_{<t})} \tag{1}$$

where $\pi_\phi$ is the PRM model, $\pi_{\text{ref}}$ is the reference model, and $\beta$ is a scaling coefficient. This enables (1) scalable online PRM update using only outcome labels on policy rollouts, (2) token-level dense rewards without step-level annotation, and (3) initialization from the SFT or base model itself, bypassing dedicated reward model training.

In the original PRIME experiments, the implicit PRM is initialized from the SFT model (Eurus-2-7B-SFT) and updated online with full parameters on 8×A800 GPUs. PRIME demonstrated a 15.1% average improvement across reasoning benchmarks starting from Qwen2.5-Math-7B-Base, with 2.5× sample efficiency gains over outcome-only rewards. Crucially, PRIME was shown to be compatible with various RL algorithms including REINFORCE, GRPO, PPO, and RLOO.

### 2.4 Our Contribution

We bridge the gap between parameter-efficient RL reasoning (Tina) and dense process rewards (PRIME) by integrating both into a unified low-resource pipeline. While PRIME was originally demonstrated with full-parameter training for both the policy and PRM, GPRA applies LoRA only to the policy model while keeping the PRM as a full-parameter model, striking a balance between memory efficiency and reward model expressiveness on a minimal two-GPU setup.

## 3 Method

### 3.1 Overview

Our approach consists of two experimental stages: (1) a LoRA+GRPO baseline that reproduces the core Tina setup, and (2) the GPRA method that augments the baseline with PRIME's implicit process rewards. Both stages use the same base model, hardware configuration, and LoRA settings for the policy model, differing only in the inclusion of the implicit PRM component. This controlled design ensures that any performance difference is directly attributable to the dense process reward mechanism.

### 3.2 Base Model and LoRA Configuration

Following Tina [Wang et al., 2025], we adopt `DeepSeek-R1-Distill-Qwen-1.5B` [DeepSeek-AI, 2025] as our base model. This model, distilled from the DeepSeek-R1 series, possesses strong initial reasoning capabilities relative to its parameter count, making it an effective starting point for evaluating the incremental benefits of RL-based post-training.

We apply LoRA adapters to the query, key, value, and dense projection layers of all transformer blocks in the **policy model only**. The LoRA configuration uses rank $r = 64$, scaling factor $\alpha = 128$, and dropout rate 0.05. All experiments use BF16 mixed-precision training with the AdamW optimizer.

### 3.3 Hardware Configuration and Training Architecture

All experiments are conducted on 2×NVIDIA A100-80GB GPUs on the RunPod cloud platform. A critical constraint motivating our hardware design is the memory requirement of GPRA: during Stage 2, the training GPU must simultaneously host the policy model (with LoRA adapters), the full-parameter implicit PRM, and their respective optimizer states for parameter updates. This combined memory footprint exceeds the capacity of a single A100-80GB GPU if we were to also run vLLM inference on the same card.

We therefore adopt a **dedicated split architecture**: one GPU is exclusively assigned to the vLLM inference engine for efficient rollout generation, while the other GPU handles all parameter updates (policy LoRA update via GRPO and full-parameter PRM update via cross-entropy loss). This architecture uses data parallelism (DP) rather than distributed data parallelism (DDP).

For the Stage 1 baseline (LoRA+GRPO without PRM), we adopt the *identical* hardware split to ensure a fair controlled comparison—even though the baseline could in principle run with DDP on two GPUs, using different parallelism strategies between stages would confound the attribution of any performance difference. We note that this DP setup results in somewhat lower absolute performance compared to multi-GPU DDP

configurations used in prior work (e.g., Tina with 2×L40S using DDP); however, our focus is on the *relative improvement* from incorporating implicit process rewards.

### 3.4 Stage 1: LoRA+GRPO Baseline

In the first stage, we train the base model using GRPO [Shao et al., 2024] with LoRA updates and outcome-only rewards. For each prompt $q$, GRPO samples a group of $G$ responses $\{o_1, o_2, \ldots, o_G\}$ from the current policy $\pi_{\theta_{\text{old}}}$ and optimizes:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\rho_i A_i,\ \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon)A_i\right) - \beta_{\text{KL}} D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\right)\right] \tag{2}$$

where $\rho_i = \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}$ is the importance ratio, and the advantage $A_i$ is computed from outcome rewards using group-level normalization:

$$A_i = \frac{r_o(y^i) - \text{mean}(\{r_o(y^j)\})}{\text{std}(\{r_o(y^j)\})} \tag{3}$$

The outcome reward $r_o$ is a rule-based verifier that checks whether the model's final answer matches the ground-truth answer in $\boxed{\cdot}$ format, combined with a format reward that encourages proper use of the `<think>...</think>` reasoning structure.

### 3.5 Stage 2: GPRA

In the second stage, we augment the GRPO pipeline with a full-parameter implicit PRM, following the PRIME framework [Cui et al., 2025]. The key additions are:

**Implicit PRM Initialization.** We initialize the implicit PRM as a **full copy of the base model** (`DeepSeek-R1-Distill-Qwen-1.5B`) without any LoRA adapters. Unlike the original PRIME which initializes the PRM from an SFT-warmed model (Eurus-2-7B-SFT), we initialize directly from the distilled base model due to the absence of a dedicated SFT stage in our pipeline. The reference model $\pi_{\text{ref}}$ is the frozen base model. The PRM is updated with **full-parameter** training (not LoRA), as the reward model requires sufficient expressiveness to provide accurate dense reward signals.

**Online Prompt Filtering.** As we sample multiple trajectories for each prompt, we introduce online prompt filtering which filters prompts within a certain accuracy range. This (1) preserves only the prompts within a certain median-level difficulty range [Yang et al., 2024] and (2) balances data distribution for the Implicit PRM online training.

**Online PRM Update.** At each training iteration, after generating rollouts and obtaining outcome rewards from the verifier, we update the implicit PRM using cross-entropy loss on the same rollouts:

$$\mathcal{L}_{\text{CE}}(\phi) = -\mathbb{E}_{(x,y,r_o(y))}\left[r_o(y)\cdot\log\sigma(r_\phi(y)) + (1-r_o(y))\cdot\log(1-\sigma(r_\phi(y)))\right] \tag{4}$$

where $r_\phi(y) = \beta\log\frac{\pi_\phi(y)}{\pi_{\text{ref}}(y)}$ is the implicit outcome reward. This update requires only outcome-level labels, which are already available from the verifier.

**Dense Advantage Estimation.** After updating the PRM, we compute token-level implicit process rewards via Eq. 1 and integrate them with outcome rewards following PRIME's advantage formulation. For compatibility with GRPO, we adapt the advantage estimate as:

$$A_t^i = \underbrace{\frac{r_o(y^i) - \text{mean}(r_o(y^j))}{\text{std}(r_o(y^j))}}_{\text{GRPO with outcome rewards}} + \underbrace{\sum_{s=t}^{|y^i|}\gamma^{s-t}\cdot\left[\frac{r_\phi(y_s^i) - \text{mean}\left(\frac{r_\phi(y^j)}{|y^j|}\right)}{\text{std}\left(\frac{r_\phi(y^j)}{|y^j|}\right)}\right]}_{\text{GRPO with implicit process rewards}} \tag{5}$$

**Algorithm 1** GPRA

**Require:** Base model $\pi_{\text{base}}$; outcome verifier $r_o$; dataset $\mathcal{D}$; sample number $K$; total iterations $N$.

1: Initialize policy model with LoRA adapters $\theta$; full-parameter PRM $\pi_\phi \leftarrow \pi_{\text{base}}$; reference $\pi_{\text{ref}} \leftarrow \pi_{\text{base}}$ (frozen)
2: **for** iteration $= 1, \ldots, N$ **do**
3: Sample batch of prompts $\mathcal{B} \sim \mathcal{D}$
4: $\mathcal{T} \leftarrow \{\}$
5: **for** each prompt $x \in \mathcal{B}$ **do**
6: Generate $K$ responses: $\{y^1, \ldots, y^K\} \sim \pi_\theta(\cdot|x)$ *[vLLM on GPU 1]*
7: Compute outcome rewards: $r_i = r_o(x, y^i)$ for $i \in \{1, \ldots, K\}$
8: Collect correct responses $\mathcal{C}_x = \{y^i \mid r_i = 1\}$
9: **if** $|\mathcal{C}_x|/K > 0.2$ **and** $|\mathcal{C}_x|/K < 0.8$ **then**
10: $\mathcal{T} \leftarrow \mathcal{T} \cup \{(x, y^i, r_i) \mid y^i \in \{y^1, \ldots, y^K\}\}$
11: **else**
12: Drop this prompt and continue
13: **end if**
14: **end for**
15: Forward pass $\pi_\phi$, $\pi_{\text{ref}}$ on each $(x, y, r) \in \mathcal{T}$ to obtain implicit process rewards $r_\phi(y_t)$ via Eq. 1 *[GPU 2]*
16: Update PRM $\pi_\phi$ (full parameters) by CE loss on $(x, y, r_o(y)) \in \mathcal{T}$ *[GPU 2]*
17: Compute advantages $A$ via Eq. 5
18: Update policy $\theta$ (LoRA only) by GRPO clipped surrogate loss *[GPU 2]*
19: **end for**
20: **return** Optimized policy model with LoRA adapters $\theta$

This formulation separates the returns from outcome rewards and process rewards to avoid numerical instability, following the design principle established in PRIME.

**Training Flow.** The complete training loop for GPRA is summarized in Algorithm 1. The critical architectural distinction from the LoRA+GRPO baseline is the addition of steps 7–8: the full-parameter PRM forward pass and online update. The policy model continues to be updated via LoRA only.

### 3.6 Design Rationale: Full-Parameter PRM with LoRA Policy

A natural question is why we apply LoRA only to the policy model while keeping the PRM as a full-parameter model. We justify this asymmetric design based on the following considerations:

1. **Reward expressiveness matters.** The implicit PRM must distinguish between correct and incorrect reasoning at the token level. Full-parameter updates provide the PRM with sufficient capacity to capture fine-grained quality signals across diverse mathematical domains. This is especially important given that our PRM is initialized from the raw distilled base model without SFT pre-training.

2. **Policy efficiency via LoRA is validated.** Tina [Wang et al., 2025] convincingly demonstrated that LoRA is highly effective for policy adaptation in RL reasoning, achieving performance competitive with full-parameter training. The "rapid format adaptation" hypothesis suggests that the policy primarily needs to learn the structural format of reasoning, which LoRA handles well.

3. **Memory feasibility on two GPUs.** With one GPU dedicated to vLLM inference, the training GPU must host both the policy (base model + LoRA) and the PRM (full model + optimizer states). Since LoRA adds only a small number of parameters to the policy, the remaining memory budget can accommodate the full-parameter PRM and its Adam optimizer states. This would not be feasible if both models required full-parameter training with optimizers.

Table 1: Training hyperparameters.

| **Hyperparameter** | **Value** |
|---|---|
| Base Model | DeepSeek-R1-Distill-Qwen-1.5B |
| *Policy Model (LoRA)* | |
| LoRA Rank ($r$) | 64 |
| LoRA Alpha ($\alpha$) | 128 |
| LoRA Dropout | 0.05 |
| LoRA Target Modules | query, key, value, dense |
| Learning Rate | $1 \times 10^{-6}$ |
| *Implicit PRM (Full Parameters)* | |
| Update Mode | Full-parameter |
| Learning Rate | $1 \times 10^{-6}$ |
| $\beta$ (reward scaling) | 0.05 |
| *General* | |
| Algorithm | GRPO |
| Optimizer | AdamW ($\beta_1$=0.9, $\beta_2$=0.999) |
| Precision | BF16-mixed |
| Number of Generations ($K$) | 4 |
| Max Prompt Length | 512 |
| Max Completion Length | 3584 |
| Hardware | 2× A100-80GB (RunPod) |
| GPU 1 | vLLM inference engine |
| GPU 2 | Policy LoRA + PRM full-param update |

# 4 Experiments

## 4.1 Experimental Setup

**Training Dataset.** We use the Open-RS dataset [Dang and Ngo, 2025], a curated subset of high-quality mathematical reasoning problems further filtered from the s1 [Muennighoff et al., 2025] and DeepScaleR [Luo et al., 2025] datasets. This dataset was selected based on Tina's finding that it achieves the best reasoning performance despite being one of the smallest datasets (approximately 7k examples).

**Hyperparameters.** We adopt the default hyperparameter settings from the Tina framework (Table 1), keeping them fixed across both the baseline and GPRA experiments. For the implicit PRM, we use a learning rate of $1 \times 10^{-6}$ and $\beta = 0.05$ following PRIME's default settings.

**Evaluation Benchmarks.** We evaluate on four mathematical reasoning benchmarks:

- **AIME 2024/2025** [Art of Problem Solving, 2024]: 30 problems each from the American Invitational Mathematics Examination, featuring challenging algebra, geometry, number theory, and combinatorics problems requiring precise multi-step reasoning.
- **AMC 2023** [Art of Problem Solving, 2023]: 40 problems from the American Mathematics Competition, testing logic and symbolic manipulation.
- **MATH-500** [Hendrycks et al., 2021, Lightman et al., 2023]: 500 competition mathematics problems of varying difficulty levels, requiring multi-step derivation and calculation.

All evaluations use zero-shot Pass@1 with consistent inference parameters (temperature=0.6, top_p=0.95, max_new_tokens=32768).

Table 2: LoRA+GRPO results (baseline). Training on 2×A100-80GB with DP.

| Step | AIME24 | AIME25 | AMC23 | MATH500 | Average |
|---|---|---|---|---|---|
| 250 | 26.67 | 16.67 | **62.50** | **78.00** | 45.96 |
| 300 | **36.67** | 20.00 | 60.00 | 73.20 | **47.46** |
| 350 | 26.67 | 20.00 | 50.00 | 76.00 | 43.17 |
| 400 | 20.00 | **23.33** | 57.50 | 75.80 | 44.16 |
| 450 | 33.33 | 13.33 | 55.00 | 73.20 | 43.72 |
| 500 | 26.67 | 20.00 | 60.00 | 75.40 | 45.52 |
| 550 | 20.00 | 20.00 | **62.50** | 75.00 | 44.38 |

Table 3: GPRA results (our method). Training on 2×A100-80GB with DP.

| Step | AIME24 | AIME25 | AMC23 | MATH500 | Average |
|---|---|---|---|---|---|
| 150 | 33.33 | 23.33 | 67.50 | 75.40 | 49.89 |
| 200 | **40.00** | 20.00 | 52.50 | 75.60 | 47.02 |
| 250 | 36.67 | 23.33 | 62.50 | 78.00 | 51.38 |
| 300 | 26.67 | **30.00** | **72.50** | **79.60** | **52.19** |
| 350 | 26.67 | 10.00 | 60.00 | 76.80 | 43.37 |
| 400 | 30.00 | 23.33 | 67.50 | 75.80 | 49.16 |
| 450 | 26.67 | 26.67 | 62.50 | 76.40 | 48.06 |

## 4.2 Main Results

Table 2 and Table 3 present the evaluation results for the LoRA+GRPO baseline and GPRA across training steps. Bold values indicate the best result in each column.

**Summary Comparison.** Table 4 provides a direct comparison of the best checkpoints from each method alongside reference baselines from the literature.

## 4.3 Key Observations

**Consistent improvement from dense rewards.** GPRA achieves a best average accuracy of 52.19% at step 300, compared to the LoRA+GRPO baseline's best of 47.46% at the same step, representing a **4.73 percentage point improvement**. This demonstrates that dense process rewards from a full-parameter implicit PRM provide meaningful benefits even when the policy is trained with LoRA.

**Improved sample efficiency.** GPRA reaches strong performance earlier in training. At step 150, GPRA already achieves 49.89% average accuracy, exceeding the baseline's peak performance of 47.46% achieved at step 300. This roughly 2× acceleration in reaching comparable performance is consistent with the 2.5× sample efficiency gain reported by PRIME [Cui et al., 2025] in the full-parameter setting.

**Stronger performance on AMC23 and MATH-500.** GPRA shows particularly strong improvements on AMC23 (72.50 vs. 60.00, +12.50) and MATH-500 (79.60 vs. 73.20, +6.40) at the best checkpoint. The MATH-500 result of 79.60 is notably approaching the full-parameter DeepScaleR-1.5B-Preview's score of 87.80, despite our model using LoRA policy updates with DP on only 2 GPUs.

**AIME25 improvement.** GPRA achieves 30.00% on AIME 2025 at step 300, a 10-point improvement over the baseline's 20.00%. This suggests that dense process rewards help the model develop more robust multi-step reasoning strategies on challenging competition problems.

Table 4: Comparison of best checkpoints and reference baselines. Results for reference models are sourced from their respective publications. †Our experiments under DP constraints.

| Model | Step | AIME24 | AIME25 | AMC23 | MATH500 | Avg. |
|---|---|---|---|---|---|---|
| *Reference baselines (from literature)* | | | | | | |
| DeepSeek-R1-Distill-Qwen-1.5B | – | 23.33 | 16.67 | 62.50 | 79.60 | 46.19 |
| Eurus-2-7B-Prime | 240 | 20.0 | 16.67 | 50.6 | 78.2 | 41.37 |
| Llama-3.1-70B-Inst. | – | 20.0 | 23.33 | 52.0 | 65.0 | 40.02 |
| *Our experiments (2×A100-80GB, DP)* | | | | | | |
| LoRA+GRPO† | 300 | 36.67 | 20.00 | 60.00 | 73.20 | 47.46 |
| GPRA† | 300 | 26.67 | 30.00 | 72.50 | 79.60 | **52.19** |
| **Improvement** | – | – | +10.00 | +12.50 | +6.40 | **+4.73** |

**High variance in competition benchmarks.** Both methods exhibit substantial variance across training steps on AIME24 and AIME25, which is expected given the small number of test problems (30 each). This variance is a known challenge in evaluating mathematical reasoning models on competition-level benchmarks [Wang et al., 2025, Luo et al., 2025].

## 4.4 Training Dynamics

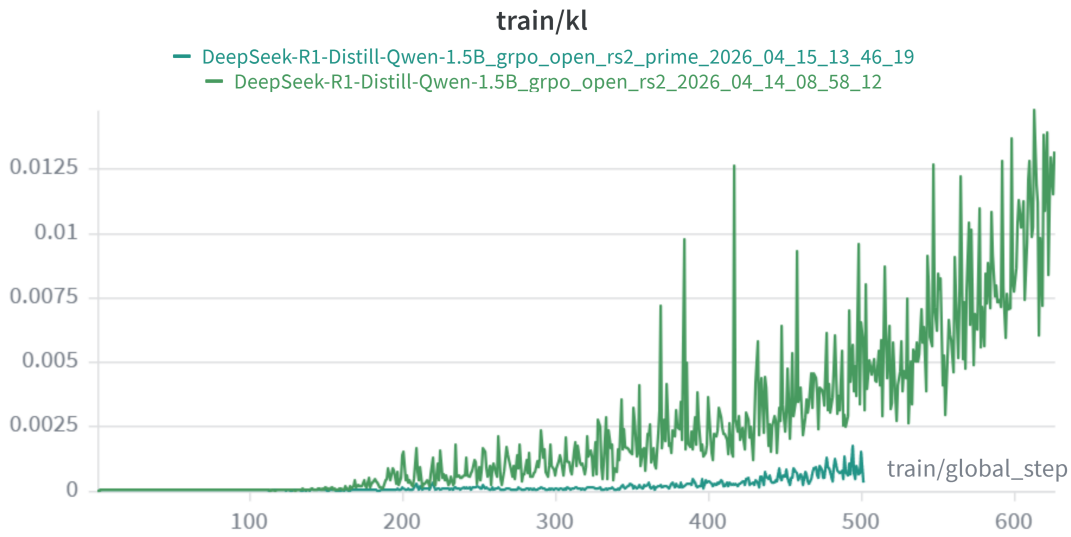


Figure 1: KL divergence between the policy and reference model during training. The darker line represents GPRA and the lighter line represents LoRA+GRPO. Despite achieving higher downstream accuracy, GPRA maintains substantially lower KL divergence throughout training. This suggests that dense process rewards provide more targeted gradient signals, enabling the policy to improve reasoning with smaller distributional shifts—learning *what to change* more precisely so it needs to change *less overall*—thereby reducing the risk of reward hacking and overoptimization.

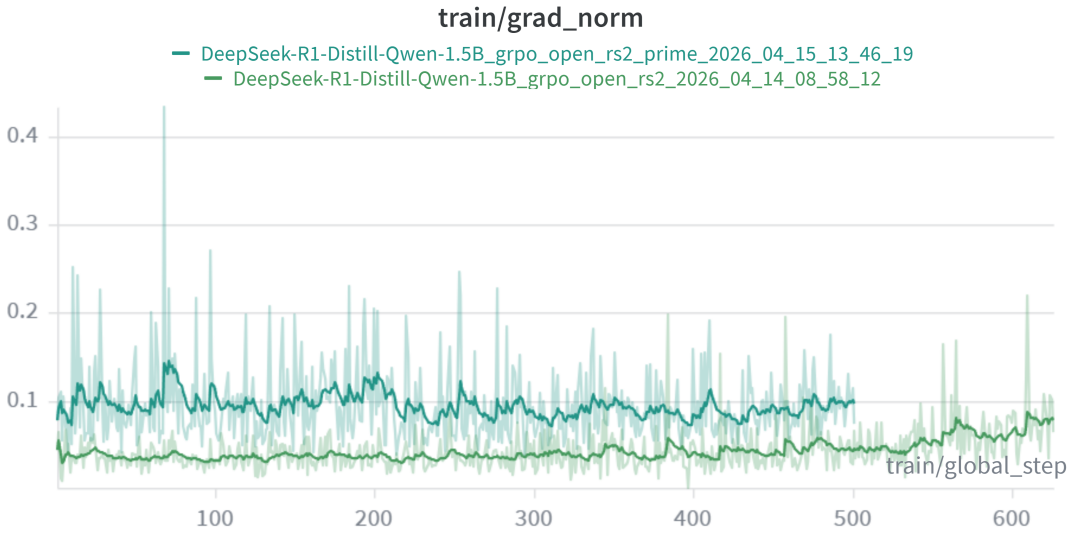


Figure 2: Gradient norm of the policy model during training. The darker line represents GPRA and the lighter line represents LoRA+GRPO. GPRA exhibits consistently larger gradient norms, which may appear contradictory to its lower KL divergence (Figure 1). However, this reflects the nature of dense process rewards: token-level reward signals produce stronger but more *precisely directed* gradients, enabling efficient policy improvement along a direct path. In contrast, the baseline's smaller but less targeted gradients from outcome-only rewards cause the policy to drift further in distribution space through indirect exploration, explaining the co-occurrence of larger gradients with lower KL in GPRA.

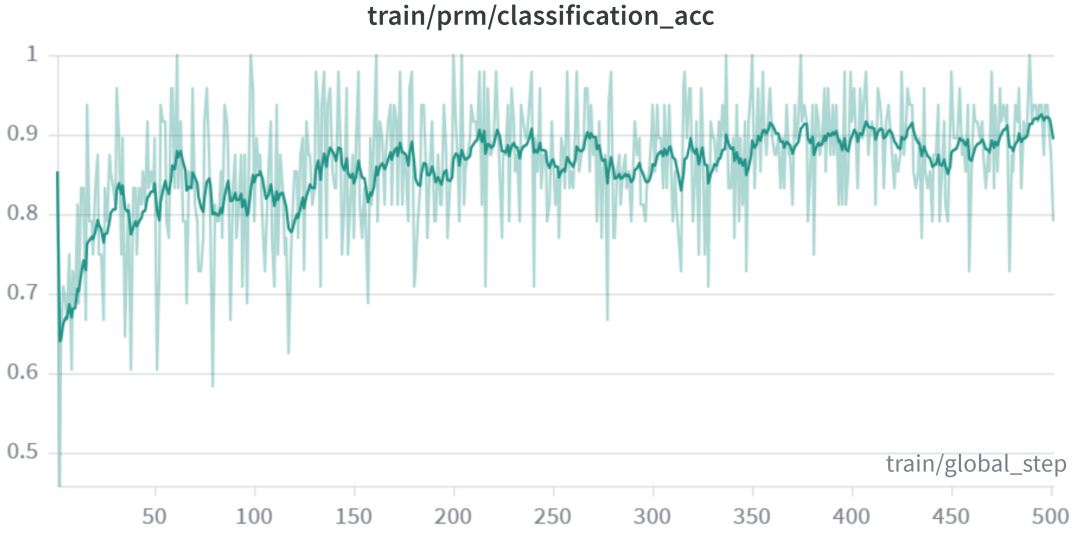


Figure 3: Implicit PRM training dynamics. The PRM classification accuracy on training rollouts is expected to increase steadily as the full-parameter PRM learns to distinguish correct from incorrect responses, validating the effectiveness of online PRM update.

# 5 Analysis and Discussion

## 5.1 Why Dense Rewards Help with LoRA Policy Updates

The improvement from GPRA over LoRA+GRPO can be understood through the lens of credit assignment. With outcome-only rewards in GRPO, the advantage signal $A_i$ is identical for all tokens within a response—a correct response receives uniform positive advantage regardless of which reasoning steps contributed to the correct answer, and vice versa for incorrect responses. This is particularly problematic for mathematical reasoning, where a single error in one step can invalidate an otherwise correct chain of reasoning.

By incorporating token-level implicit process rewards from the full-parameter PRM, GPRA provides differentiated feedback across the response. Tokens that contribute to correct reasoning patterns receive higher process rewards, while erroneous or redundant steps receive lower rewards. This finer-grained signal enables the LoRA adapters in the policy model to make more targeted updates, improving the efficiency of the parameter-efficient optimization.

## 5.2 Full-Parameter PRM: Expressiveness for Dense Rewards

An important design decision in GPRA is using full-parameter updates for the PRM while restricting the policy to LoRA. This asymmetry is justified by the distinct roles of the two models: the policy primarily needs to adapt its output *format* to produce structured reasoning chains (well-suited to LoRA, per Tina's hypothesis), while the PRM must develop fine-grained discriminative ability over token-level reasoning quality, requiring greater model capacity.

This design is further supported by PRIME's finding that the PRM is initialized from the SFT model and trained from scratch on online rollouts. In our case, the PRM starts from the distilled base model and must learn entirely from the reward signal, making full-parameter expressiveness crucial for effective online adaptation.

## 5.3 Comparison with PRIME's GRPO Results

Comparing with PRIME's original results on the 7B Qwen2.5-Math model (Table 4 in Cui et al. [2025]), the GRPO+PRIME combination achieved a 1.7% average improvement over GRPO alone at 240 steps (37.8 vs. 36.1). Our 4.73% improvement with GPRA over LoRA+GRPO suggests that the dense reward signal may be particularly beneficial when combined with parameter-efficient policy updates, where the limited expressiveness of LoRA adapters can be better guided by fine-grained reward signals from a full-parameter PRM.

## 5.4 Limitations and Future Work

Our work has several limitations that suggest directions for future investigation:

1. **Hardware constraints and training architecture.** Our DP setup with one GPU for inference and one for training is suboptimal compared to multi-GPU DDP configurations. Scaling to more GPUs would likely improve absolute performance for both methods and could enable larger batch sizes for more stable advantage estimation.

2. **PRM initialization without SFT.** In the original PRIME framework, the implicit PRM is initialized from the SFT model (Eurus-2-7B-SFT), which has already been fine-tuned on reasoning data and thus provides a stronger starting point for reward modeling. In contrast, our PRM is initialized directly from the distilled base model without a dedicated SFT stage. As PRIME demonstrated that SFT initialization largely alleviates distribution shift between the PRM and the policy, incorporating an SFT warmup stage for the PRM could further improve GPRA's performance. This is a promising direction for future work.

3. **Evaluation scope.** We evaluate on four mathematical benchmarks. Extending to coding tasks, scientific reasoning (GPQA, Minerva), and general reasoning would provide a more comprehensive assessment of GPRA's generalizability.

4. **Hyperparameter sensitivity.** We adopted default hyperparameters from Tina and PRIME without task-specific tuning. Systematic exploration of the PRM learning rate, $\beta$, discount factor $\gamma$, and LoRA rank could further improve results.

5. **Training stability.** GPRA shows a performance drop at step 350 (43.37%), suggesting potential instability in the PRM update dynamics. Investigating techniques such as gradient clipping, PRM update frequency reduction, or EMA-based PRM smoothing could help stabilize training. This instability may also be partially related to the lack of SFT pre-training for the PRM, as the reward signal from a randomly-initialized PRM may introduce noise in the early stages of training.

6. **Scaling to larger models.** Exploring whether GPRA scales to 7B or larger models would be valuable, particularly given PRIME's finding that larger models benefit more from RL training [DeepSeek-AI, 2025].

# 6 Conclusion

We presented GPRA (GRPO + PRIME + LoRA), a resource-efficient approach to enhancing mathematical reasoning in small language models that combines LoRA-based parameter-efficient policy optimization with full-parameter online implicit process rewards. GPRA achieves a 4.73 percentage point improvement over the LoRA+GRPO baseline across four mathematical reasoning benchmarks, while demonstrating improved sample efficiency and stronger performance on competition-level problems—all on a minimal hardware budget of two A100 GPUs.

Our key findings are twofold. First, *dense process rewards remain effective when the policy model is trained with LoRA*, providing meaningful improvements in credit assignment and training efficiency. Second, the *asymmetric design*—LoRA for the policy, full parameters for the PRM—offers a practical balance between memory efficiency and reward model expressiveness that enables the PRIME mechanism to function on resource-constrained hardware.

We believe this work contributes to the broader goal of democratizing reasoning model development. By showing that significant reasoning improvements can be achieved with minimal resources through the judicious combination of parameter efficiency (LoRA), algorithmic efficiency (GRPO), and reward efficiency (implicit PRMs), we hope to enable wider participation in the exploration of RL-based reasoning enhancement. All code, training logs, and model weights are publicly available to facilitate reproduction and further research.

# References


Art of Problem Solving. AMC problems and solutions, 2023. URL https://artofproblemsolving.com/wiki/index.php/AMC_12_Problems_and_Solutions.

Art of Problem Solving. AIME problems and solutions, 2024. URL https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.

Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.

Quy-Anh Dang and Chris Ngo. Reinforcement learning for reasoning in small LLMs: What works and what doesn't. *arXiv preprint arXiv:2503.16219*, 2025.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *Proceedings of ICLR*, 2023.

Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. DeepScaleR: Surpassing o1-preview with a 1.5B model by scaling RL. 2025.

Yingqian Min, Zhipeng Chen, Jinhao Jiang, Jie Chen, Jia Deng, Yiwen Hu, Yiru Tang, Jiapeng Wang, Xiaoxue Cheng, Huatong Song, Wayne Xin Zhao, Zheng Liu, Zhongyuan Wang, and Ji-Rong Wen. Imitate, explore, and self-improve: A reproduction report on slow-thinking reasoning systems. *arXiv preprint arXiv:2412.09413*, 2024.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

OpenAI. OpenAI o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Y.Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. *arXiv preprint arXiv:2312.08935*, 2023.

Shangshang Wang, Julian Asilis, Ömer Faruk Akgül, Enes Burak Bilgin, Ollie Liu, and Willie Neiswanger. Tina: Tiny reasoning models via LoRA. *arXiv preprint arXiv:2504.15777*, 2025.

Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. *arXiv preprint arXiv:2412.01981*, 2024.

Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. SimpleRL-Zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.